

Apostila 02

Segmentação, Morfologia e Análise de Formas

Trilha 2 — Semanas 4 a 6 · Do pixel filtrado ao objeto medido

Unidade 2.1

Limiarização e Morfologia
Matemática

Unidade 2.2

Contornos, Descritores de
Forma e Hough

Unidade 2.3

Domínio da Frequência e
Contornos Ativos

Como usar esta apostila

Este material foi escrito para ser lido sozinho, sem depender de uma aula ao vivo.

Cada conceito aparece três vezes: como ideia, como matemática e como código executável.

1

Leia a ideia

Comece pela intuição de cada seção. Se a intuição não fizer sentido, a fórmula também não fará.

2

Confira a matemática

Os blocos em destaque trazem a formulação. Você não precisa decorar: precisa saber o que cada símbolo controla.

3

Execute o código

Todo bloco em fonte monoespçada roda no Colab da trilha. Altere um parâmetro por vez e observe o efeito.

4

Aplique na AED

Ao final de cada unidade há uma ponte com o Projeto Integrador. É ali que o conteúdo vira entrega.

Antes de começar, confirme que você domina a Trilha 1

Tudo aqui parte de uma imagem já convertida para tons de cinza e já suavizada. Se `cv2.cvtColor`, `cv2.GaussianBlur` e `cv2.Canny` ainda soam estranhos, releia a Apostila 01 — segmentar uma imagem ruidosa produz resultados que parecem certos no monitor e falham no relatório.

Onde estamos: o mapa da disciplina

A disciplina tem uma progressão deliberada: primeiro você aprende a limpar a imagem, depois a extrair objetos dela e só então a ensinar uma rede neural a reconhecê-los. Pular etapas produz modelos que aprendem ruído.

TRILHA 1 — CONCLUÍDA

Fundamentos e filtragem

- Imagem como matriz NumPy
- Espaços de cor BGR / HSV
- Convolução e filtros espaciais
- Bordas: Sobel, Laplaciano, Canny

TRILHA 2 — VOCÊ ESTÁ AQUI

Do pixel ao objeto

- Limiarização e Otsu
- Morfologia matemática
- Contornos e descritores de forma
- Hough, Fourier e Snakes

TRILHA 3 — A SEGUIR

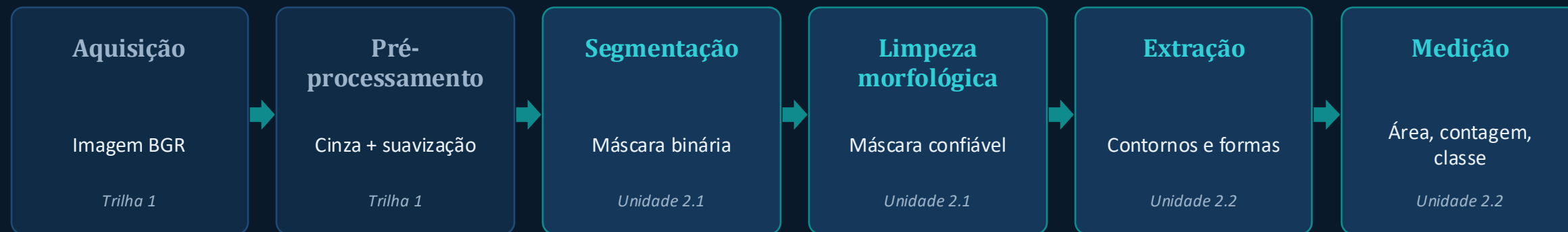
Rotulagem e fechamento da N1

- Consistent Labelling
- Atividade Estruturada 1 (Ex1)
- Defesa do PP1
- Avaliação individual P1

Regra prática: um detector moderno como o YOLO não elimina o pré-processamento — ele apenas o esconde. Quem entende esta trilha depura o que a rede não explica.

O fio condutor: da imagem crua ao objeto medido

Toda a Trilha 2 responde a uma única pergunta de engenharia: como transformar uma fotografia em números confiáveis sobre os objetos que aparecem nela?



O erro que custa a nota

Estudantes tendem a caprichar no modelo e improvisar a segmentação. Na prática industrial acontece o contrário: 70% do esforço de um sistema de inspeção está em obter uma máscara estável sob iluminação variável. É esse esforço que a Trilha 2 ensina.

O que muda na Unidade 2.3

As unidades 2.1 e 2.2 trabalham no domínio espacial — pixels e vizinhanças. A unidade 2.3 troca de lente: olha a mesma imagem como soma de frequências (Fourier e Wavelets) e como curva que se deforma até encaixar no objeto (Snakes).

O que você será capaz de fazer ao final da trilha

1 Escolher o limiar certo

Decidir entre limiarização global, adaptativa e Otsu justificando a escolha pelo histograma e pela iluminação da cena.

2 Limpar máscaras binárias

Aplicar erosão, dilatação, abertura, fechamento e top-hat para remover ruído sem destruir a geometria do objeto.

3 Extrair e medir objetos

Obter contornos, hierarquia, área, perímetro, centróide e caixas delimitadoras a partir de uma máscara.

4 Classificar formas sem IA

Usar circularidade, solidez e momentos de Hu para separar categorias simples de objetos.

5 Detectar estruturas geométricas

Localizar retas e círculos com a Transformada de Hough e corrigir perspectiva com homografia.

6 Ler a imagem em frequência

Interpretar o espectro de Fourier, filtrar ruído periódico e reconhecer quando Wavelets são a ferramenta adequada.

Esses seis objetivos são exatamente o que a Avaliação P1 (06/10/2026) e a Atividade Estruturada 1 (25/09/2026) cobram.

Roteiro da Trilha 2

UNIDADE 2.1

Limiarização e Morfologia Matemática

- Limiar global, adaptativo e método de Otsu
- Elemento estruturante e operações de conjunto
- Erosão, dilatação, abertura, fechamento
- Gradiente morfológico, top-hat e black-hat

01/09 e 04/09

UNIDADE 2.2

Contornos, Descritores de Forma e Hough

- findContours, hierarquia e aproximação
- Momentos, área, perímetro e centróide
- Circularidade, solidez e momentos de Hu
- Hough para retas e círculos; retificação de perspectiva

08/09 e 11/09

UNIDADE 2.3

Domínio da Frequência e Contornos Ativos

- Transformada Discreta de Fourier em imagens
- Filtragem passa-baixa e passa-alta no espectro
- Introdução a Wavelets e análise multiescala
- Snakes e representação interna de objetos

15/09 e 18/09

As datas indicam a referência do cronograma oficial da disciplina. Como a oferta é 100% assíncrona, elas marcam a abertura do conteúdo — não um horário fixo de encontro.

2.1

Limiarização e Morfologia Matemática

Como decidir, pixel a pixel, o que é objeto e o que é fundo — e depois consertar a decisão

Segmentação

Global, adaptativa e Otsu

Morfologia

Erosão, dilatação, abertura,
fechamento

Aplicação

Máscaras estáveis para
contagem e inspeção

Por que binarizar uma imagem?

Uma imagem em tons de cinza tem 256 respostas possíveis por pixel. Um sistema de inspeção precisa de duas: "isto pertence ao objeto" ou "isto é fundo". Binarizar é o ato de tomar essa decisão.

- Reduz o problema.
- Uma máscara binária ocupa 1 bit por pixel e permite operações de conjunto (união, interseção, diferença) que são baratas e previsíveis.
- Habilita a etapa seguinte. `cv2.findContours`, o cálculo de área e a contagem de objetos só funcionam sobre imagens binárias.
- Torna o sistema auditável. Diferente de uma rede neural, cada decisão vem de um número que você escolheu e pode justificar no relatório técnico.
- É rápido o bastante para tempo real. Uma limiarização em Full HD custa menos de 1 ms em CPU comum.

A operação em uma linha

$$b(x, y) = \begin{cases} 1, & \text{se } I(x, y) \geq T \\ 0, & \text{caso contrário} \end{cases}$$

Toda a dificuldade da segmentação clássica cabe em uma pergunta: quem escolhe o T , e com base em quê?

Três respostas possíveis:

- Você escolhe → limiar global
- A estatística escolhe → método de Otsu
- Cada vizinhança escolhe a sua → limiarização adaptativa

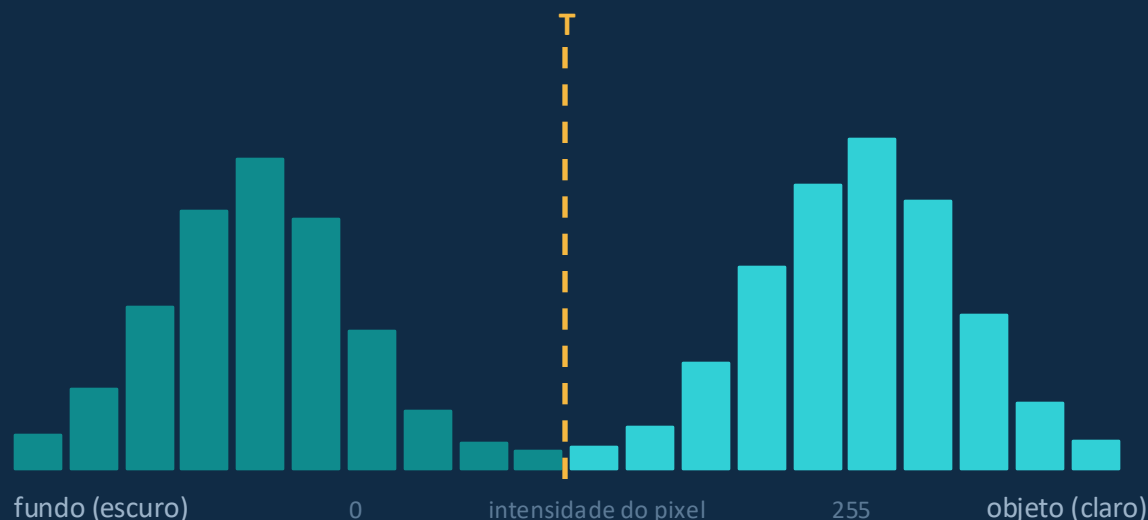
Nesta trilha você aprende as três — e, mais importante, quando cada uma falha.

O histograma é o mapa onde o limiar é escolhido

O histograma conta quantos pixels existem em cada nível de intensidade.

Ele não diz onde os pixels estão, mas diz se existe uma separação possível entre objeto e fundo.

Histograma bimodal — o caso favorável



Leitura de um histograma bimodal

Dois picos separados por um vale significam duas populações de pixels. O limiar ideal fica no fundo do vale: ali há o menor número de pixels ambíguos, portanto o menor número de erros de classificação.

Quando não há vale

Histograma unimodal (um único pico) indica que objeto e fundo têm brilhos parecidos. Nenhum limiar global resolverá isso — o caminho é mudar a iluminação, trocar de canal (por exemplo, o canal S do HSV) ou partir para limiarização adaptativa.

INSPEÇÃO OBRIGATÓRIA ANTES DE SEGMENTAR

```
hist = cv2.calcHist([cinza], [0], None, [256], [0, 256])  
plt.plot(hist); plt.title("Histograma de intensidades")
```

```
# 256 níveis  
# inspecione ANTES de escolher T
```

Limiarização global: o limiar que você escolhe

Um único valor T é aplicado a todos os pixels da imagem. É a técnica mais simples, mais rápida e a mais frágil — funciona muito bem em ambientes controlados e falha assim que a iluminação varia.

Tipo (flag do OpenCV)	O que faz com o pixel	Quando usar
THRESH_BINARY	Vira maxval se $I > T$; senão vira 0	Caso padrão: objeto claro sobre fundo escuro
THRESH_BINARY_INV	Vira 0 se $I > T$; senão vira maxval	Objeto escuro sobre fundo claro — texto preto em papel branco
THRESH_TRUNC	Satura em T se $I > T$; senão mantém I	Limitar brilhos extremos sem binarizar (reduz reflexos)
THRESH_TOZERO	Mantém I se $I > T$; senão vira 0	Preservar a intensidade original apenas das regiões relevantes
THRESH_TOZERO_INV	Vira 0 se $I > T$; senão mantém I	Uso inverso do anterior, menos frequente na prática

LIMIARIZAÇÃO GLOBAL EM OPENCV

```
T = 127                                     # valor escolhido por você
ret, binaria = cv2.threshold(cinza, T, 255, cv2.THRESH_BINARY)
# ret devolve o limiar efetivamente usado; binaria e a mascara
# resultante
```

Cuidado com o "127 mágico"

Escolher 127 porque é a metade de 255 é um chute, não um método. Se a média da sua imagem for 40 ou 210, esse limiar simplesmente apaga tudo ou aceita tudo.

Quando o limiar fixo quebra (e ele quebra sempre)

Iluminação não uniforme

Uma lâmpada lateral deixa um lado da peça mais claro que o outro. Um único T classifica a metade escura do objeto como fundo.

Saída: Limiarização adaptativa ou correção prévia com top-hat

Variação entre lotes

O limiar calibrado hoje deixa de servir amanhã, quando a esteira recebe peças de outro fornecedor com acabamento diferente.

Saída: Otsu, que recalcula T para cada imagem

Baixo contraste

Trincas finas em metal escuro produzem diferença de poucos níveis de cinza. Nenhum limiar separa o que o sensor mal registrou.

Saída: Equalização de histograma ou CLAHE antes de segmentar

Reflexos especulares

Um brilho saturado vira um "objeto" branco que não existe, inflando a contagem.

Saída: Filtro polarizador na captura ou remoção por área mínima

Nenhum desses problemas é resolvido por um modelo maior. Todos são resolvidos por um pipeline melhor.

Método de Otsu: deixar a estatística escolher o limiar

Publicado por Nobuyuki Otsu em 1979, o método testa exaustivamente todos os limiares possíveis e escolhe aquele que produz a melhor separação entre as duas populações de pixels. É a resposta automática para o "quem escolhe o T".

1 Percorre todos os T

Para cada valor de 0 a 255, divide os pixels em dois grupos: abaixo e acima do limiar.

2 Mede a homogeneidade

Calcula a variância dentro de cada grupo, ponderada pelo número de pixels que cada um contém.

3 Escolhe o melhor corte

Fica com o T que minimiza a variância intraclasse — o que equivale a maximizar a variância interclasse.

A formulação

$$\sigma^2_{\text{intraclasse}}(T) = \omega_0(T) \cdot \sigma_0^2(T) + \omega_1(T) \cdot \sigma_1^2(T)$$

$$\sigma^2_{\text{interclasse}}(T) = \omega_0(T) \cdot \omega_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$$

ω = proporção de pixels do grupo · μ = média do grupo · σ^2 = variância do grupo.

Minimizar a primeira expressão é equivalente a maximizar a segunda — por isso as duas definições que você encontra na literatura levam ao mesmo T.

A condição que Otsu exige

O método pressupõe um histograma bimodal. Em imagens com iluminação irregular, muito ruído ou objetos que ocupam menos de 5% da área, o "vale" não existe e Otsu devolve um limiar sem sentido — sem avisar. Sempre valide o resultado visualmente.

Otsu na prática: duas linhas e uma armadilha

SEGMENTAÇÃO AUTOMÁTICA COM OTSU

```
# A suavizacao previa NAO e opcional: Otsu e sensivel a ruído
suave = cv2.GaussianBlur(cinza, (5, 5), 0)

# O limiar informado (o zero) e ignorado quando THRESH_OTSU esta ativo
T, mascara = cv2.threshold(suave, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Limiar calculado automaticamente:", T)
```

Por que somar as duas flags

THRESH_BINARY define o formato da saída (0 ou 255). THRESH_OTSU define de onde vem o limiar. Somar as duas diz ao OpenCV: "binarize, mas calcule você mesmo o corte". O valor 0 passado como limiar é apenas um marcador de posição.

Sempre imprima o T retornado

O valor calculado é a evidência técnica que vai para o relatório da AED. Se ele varia muito entre imagens do mesmo lote, sua iluminação está instável.

Otsu não é adaptativo

Ele calcula um único limiar para a imagem inteira. Iluminação em gradiente continua sendo um problema — a solução é a limiarização adaptativa.

Otsu funciona por região

Se você já sabe onde o objeto está, aplique Otsu apenas naquela ROI. A bimodalidade local costuma ser muito mais nítida que a global.

Limiarização adaptativa: um limiar para cada vizinhança

Em vez de um T para a imagem inteira, calcula-se um T para cada pixel a partir da estatística de uma janela ao redor dele. É a técnica que salva documentos fotografados com sombra e peças iluminadas de lado.

Parâmetro	O que significa	Como ajustar na prática
adaptiveMethod	ADAPTIVE_THRESH_MEAN_C usa a média da janela; GAUSSIAN_C usa média ponderada por gaussiana	GAUSSIAN_C produz bordas mais suaves e é o padrão recomendado para texto
blockSize	Tamanho da janela em pixels. Precisa ser ímpar e maior que 1	Deve caber o objeto e um pouco de fundo: 11 a 31 para texto; maior para peças grandes
C	Constante subtraída da média local, ajusta a sensibilidade	Aumente se aparecer ruído no fundo; diminua se o objeto estiver sendo apagado
maxValue	Valor atribuído aos pixels aprovados	Praticamente sempre 255

LIMIARIZAÇÃO ADAPTATIVA GAUSSIANA

```
adapt = cv2.adaptiveThreshold(cinza, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 21, 5)
# blockSize=21, C=5
```

Regra de bolso: blockSize grande demais faz a adaptativa se comportar como global; pequeno demais transforma a textura interna do objeto em contorno falso.

Global × Otsu × Adaptativa: escolhendo com critério

Critério	Global (T fixo)	Otsu	Adaptativa
Quem define o limiar	Você, manualmente	A estatística do histograma	A vizinhança de cada pixel
Custo computacional	Mínimo	Baixo (uma varredura do histograma)	Moderado (uma janela por pixel)
Tolera iluminação irregular	Não	Não	Sim — é exatamente o seu propósito
Exige histograma bimodal	Não, mas exige calibração	Sim	Não
Reprodutibilidade entre lotes	Baixa: precisa recalibrar	Alta: recalcula por imagem	Alta, se os parâmetros forem estáveis
Cenário típico	Bancada com iluminação controlada	Esteira com fundo uniforme	Documentos, placas, fotos de celular

A decisão em uma frase

Se a iluminação é controlada, comece pelo Otsu — ele é automático e auditável. Se a imagem chega de um celular, de uma câmera móvel ou de um ambiente aberto, use adaptativa. O limiar global fixo só se justifica quando você controla fisicamente a cena e precisa de determinismo absoluto.

Checklist antes de considerar a segmentação pronta

Plotei o histograma?

Sem essa inspeção, qualquer limiar é chute.

Suavizei antes de binarizar?

Gaussiano ou mediana reduzem falsos positivos de ruído.

Testei o canal certo?

Às vezes o canal S ou V do HSV separa melhor que o tom de cinza.

Registrei o T obtido?

O valor é evidência técnica para o relatório e para a defesa.

Testei em mais de uma imagem?

Um pipeline que funciona em uma foto não é um pipeline.

Validei visualmente a máscara?

Sobreponha a máscara na imagem original antes de seguir adiante.

Ponte com a AED — Checkpoint 1 (11/09/2026)

O entregável do Checkpoint 1 exige exatamente este bloco: dataset estruturado e um pipeline de pré-processamento que leia, redimensione, converta espaço de cor, filtre ruído e produza uma máscara binária. Se sua equipe já responde "sim" às seis perguntas acima, o checkpoint está cumprido.

2.1b

Morfologia Matemática

A máscara binária quase nunca sai perfeita — morfologia é a caixa de ferramentas que a conserta

Base teórica

Operações de conjunto sobre pixels

Operadores

Erosão, dilatação, abertura, fechamento

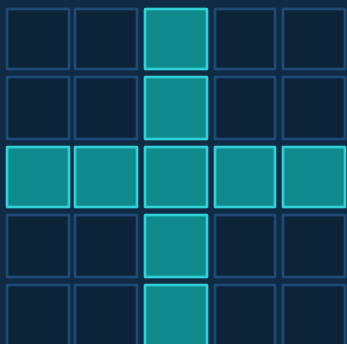
Derivados

Gradiente, top-hat e black-hat

A ideia central: forma testando forma

Morfologia matemática trata a imagem binária como um conjunto de pontos e a analisa deslizando sobre ela outro conjunto, pequeno e de forma conhecida: o elemento estruturante (kernel). A pergunta que cada operação faz é sempre a mesma — "o elemento estruturante cabe aqui?".

Elemento estruturante 5×5



MORPH_CROSS — preserva estruturas finas em cruz

Forma	Constante OpenCV	Efeito característico
Retângulo	MORPH_RECT	Padrão. Age igualmente em todas as direções da caixa
Elipse	MORPH_ELLIPSE	Preserva melhor contornos arredondados — ideal para peças e células
Cruz	MORPH_CROSS	Age só na horizontal e vertical; útil para linhas de tabelas e textos

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)) # sempre nomeie o kernel
```

O tamanho do kernel é o parâmetro que decide tudo

Kernel pequeno demais não remove o ruído; grande demais come o objeto. Regra prática: o kernel deve ser maior que o maior ruído e menor que a menor feição do objeto que você precisa preservar. Meça essas duas coisas em pixels antes de escolher.

Erosão e dilatação: os dois verbos primitivos

EROSÃO

```
cv2.erode( )
```

Encolhe as regiões brancas

- Um pixel só permanece branco se o kernel inteiro couber dentro da região branca.
- Remove pontos brancos isolados (ruído sal).
- Separa objetos que se tocam por uma ponte fina.
- Efeito colateral: reduz a área real do objeto — corrija com uma dilatação equivalente.

DILATAÇÃO

```
cv2.dilate( )
```

Expande as regiões brancas

- Um pixel vira branco se o kernel tocar qualquer pixel branco.
- Fecha buracos pequenos dentro do objeto.
- Reconecta partes de um objeto quebradas pela binarização.
- Efeito colateral: engorda o objeto e pode fundir objetos vizinhos.

AS DUAS OPERAÇÕES BÁSICAS

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
erodida = cv2.erode(mascara, k, iterations=1)      # iterations repete a operacao
dilatada = cv2.dilate(mascara, k, iterations=1)    # 2 iteracoes com k(3x3) = 1 com k(5x5)
```

Abertura e fechamento: as combinações que você usa

Erosão e dilatação isoladas alteram o tamanho do objeto. Aplicadas em sequência, elas limpam a máscara e devolvem o objeto ao tamanho aproximado original. São essas duas combinações que aparecem em praticamente todo pipeline profissional.

ABERTURA

erosão → dilatação · MORPH_OPEN

Remove ruído branco no fundo e elimina protuberâncias finas, sem encolher o objeto principal.

Use quando: pontinhos brancos poluem o fundo da máscara.

FECHAMENTO

dilatação → erosão · MORPH_CLOSE

Fecha buracos e fendas escuras dentro do objeto, sem engordar o contorno externo.

Use quando: o objeto sai furado ou com rachaduras falsas.

A SEQUÊNCIA DE LIMPEZA PADRÃO

```
aberta = cv2.morphologyEx(mascara, cv2.MORPH_OPEN, k) # tira ruido do fundo
fechada = cv2.morphologyEx(aberta, cv2.MORPH_CLOSE, k) # tapa buracos do objeto
# a ordem importa: abrir depois fechar e a sequencia padrao de limpeza
```

Gradiente, Top-hat e Black-hat: os operadores derivados

UNIDADE 2.1

GRADIENTE MORFOLÓGICO

MORPH_GRADIENT

dilatação - erosão

Resulta no contorno do objeto com espessura controlada pelo kernel. É uma alternativa ao Canny quando você já tem uma máscara binária limpa e quer apenas a borda.

TOP-HAT (WHITE HAT)

MORPH_TOPHAT

original - abertura

Realça detalhes claros menores que o kernel sobre fundo variável. É a ferramenta clássica para corrigir iluminação não uniforme ANTES de binarizar.

BLACK-HAT

MORPH_BLACKHAT

fechamento - original

Realça detalhes escuros sobre fundo claro. Usado para extrair caracteres pretos de placas e documentos com fundo irregular.

TOP-HAT COMO CORREÇÃO DE ILUMINAÇÃO

```
kg = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (25, 25))
fundo_corrigido = cv2.morphologyEx(cinza, cv2.MORPH_TOPHAT, kg)
T, mask = cv2.threshold(fundo_corrigido, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
```

Detalhe que muda o resultado

No top-hat, o kernel precisa ser MAIOR que o objeto de interesse — ele representa a escala do fundo que você quer remover. Por isso 25×25, e não 5×5.

Receita completa: da foto à máscara confiável

PIPELINE DE SEGMENTAÇÃO — UNIDADE 2.1

```
import cv2
import numpy as np

img = cv2.imread("peca.jpg")
cinza = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Corrige iluminacao de fundo (opcional, mas quase sempre ajuda)
kg = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (31, 31))
plano = cv2.morphologyEx(cinza, cv2.MORPH_TOPHAT, kg)

# 2. Atenua ruido antes de decidir o limiar
suave = cv2.GaussianBlur(plano, (5, 5), 0)

# 3. Segmenta automaticamente
T, mascara = cv2.threshold(suave, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 4. Limpa a mascara: primeiro o fundo, depois o objeto
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
limpa = cv2.morphologyEx(mascara, cv2.MORPH_OPEN, k, iterations=1)
limpa = cv2.morphologyEx(limpa, cv2.MORPH_CLOSE, k, iterations=2)

print(f"Limiar de Otsu: {T:.1f} | pixels de objeto: {int(limpa.sum()/255)}")
```

Por que top-hat antes?

Remove o gradiente de iluminação e devolve um fundo plano, condição que o Otsu precisa para funcionar.

Por que suavizar depois?

O top-hat pode acentuar ruído de alta frequência; o gaussiano corrige isso sem borrar a borda relevante.

Por que abrir antes de fechar?

Se você fechar primeiro, o ruído do fundo é incorporado ao objeto e não sai mais.

Por que imprimir o T?

Esse número entra no relatório técnico e é a métrica que denuncia instabilidade de iluminação entre imagens.

Este é o código mínimo esperado no Checkpoint 1 da AED. Adapte os tamanhos de kernel ao seu dataset e documente a escolha no README.

Erros que mais aparecem na Unidade 2.1

! Binarizar sem suavizar

Cada pixel de ruído vira um objeto. A contagem estoura e a equipe conclui, erradamente, que "Otsu não funciona".

! Usar kernel quadrado em objeto redondo

MORPH_RECT deixa cantos artificiais no contorno, o que distorce perímetro e circularidade na etapa seguinte.

! Aumentar iterations até "ficar bonito"

Cada iteração adicional afasta a máscara da geometria real. Se precisa de 6 iterações, o problema está na binarização, não na morfologia.

! Aplicar morfologia na imagem colorida

As operações são definidas para conjuntos. Em imagem BGR o resultado é imprevisível — converta para cinza ou binário antes.

! Confundir borda com contorno

O gradiente morfológico devolve pixels de borda; findContours devolve uma lista ordenada de pontos. São coisas diferentes com usos diferentes.

! Não versionar os parâmetros

Kernel, blockSize e C precisam estar no código versionado, não em um notebook que só roda na máquina de um integrante.

2.2

Contornos, Descritores de Forma e Hough

A máscara está pronta. Agora é hora de transformar pixels brancos em objetos com nome, tamanho e posição

Contornos

Extração, hierarquia e aproximação

Descritores

Área, momentos, circularidade, Hu

Hough

Retas, círculos e retificação de perspectiva

Borda e contorno: a diferença que muda o código

BORDA (edge)

Resultado de Canny, Sobel ou gradiente morfológico

- É uma imagem: uma matriz do mesmo tamanho da original.
- Cada pixel diz apenas "aqui há uma transição de intensidade".
- Não há noção de qual borda pertence a qual objeto.
- Bordas podem ser interrompidas e não formar figuras fechadas.

CONTORNO (contour)

Resultado de `cv2.findContours` sobre imagem binária

- É uma estrutura de dados: uma lista de arrays de pontos (x, y).
- Cada contorno é uma entidade — você pode medir, contar e classificar.
- Os pontos vêm ordenados ao longo da fronteira, o que permite calcular perímetro e área.
- Traz hierarquia: quem está dentro de quem.

Consequência prática

`findContours` espera uma imagem binária, não um mapa de bordas do Canny. Passar o resultado do Canny funciona por acidente em imagens simples, mas produz contornos duplicados (um para cada lado da linha de borda), o que dobra a contagem de objetos. Segmente, limpe com morfologia e só então extraia contornos.

cv2.findContours: os dois parâmetros que definem tudo

UNIDADE 2.2

Modo de recuperação	O que devolve	Quando usar
RETR_EXTERNAL	Apenas os contornos mais externos; ignora tudo que estiver dentro	Contagem de objetos — é o modo mais usado em inspeção
RETR_LIST	Todos os contornos, sem relação de hierarquia entre eles	Quando você quer tudo e não se importa com quem está dentro de quem
RETR_CCOMP	Organiza em dois níveis: fronteiras externas e buracos	Detecção de furos em peças (a furação é o nível interno)
RETR_TREE	Árvore completa de aninhamento	Estruturas encaixadas: caracteres com contra-forma, placas dentro de molduras

Método de aproximação	Efeito
CHAIN_APPROX_NONE	Guarda absolutamente todos os pontos da fronteira — mais memória, útil se você precisa da curva completa
CHAIN_APPROX_SIMPLE	Comprime segmentos retos em seus extremos: um retângulo passa de centenas de pontos para 4. Padrão recomendado

EXTRAÇÃO DE CONTORNOS

```
contornos, hierarquia = cv2.findContours(mascara,  
                                         cv2.RETR_EXTERNAL,  
                                         cv2.CHAIN_APPROX_SIMPLE)  
print("Objetos encontrados:", len(contornos))    # a contagem sai daqui
```

Hierarquia: descobrindo quem está dentro de quem

Para cada contorno, o OpenCV devolve um vetor de quatro inteiros. Eles descrevem a posição daquele contorno na árvore de aninhamento. O valor -1 significa "não existe".

[0] Next

Próximo contorno no mesmo nível

[1] Previous

Contorno anterior no mesmo nível

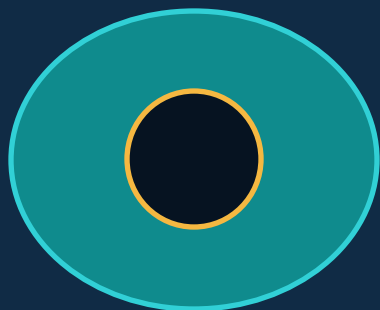
[2] First_Child

Primeiro contorno aninhado dentro deste

[3] Parent

Contorno que contém este

Exemplo: uma arruela sobre a mesa



contorno externo (Parent = -1)

furo interno (Parent = 0)

Para que serve na prática

Detectar furos, contar apenas objetos externos e ignorar texturas internas. Em inspeção de peças, um objeto sem filho pode significar furação ausente — um defeito detectável sem nenhuma rede neural.

```
cnts, hier = cv2.findContours(mask, cv2.RETR_CCOMP,
cv2.CHAIN_APPROX_SIMPLE)
furos = [c for i, c in enumerate(cnts) if hier[0][i][3] != -1]
print("Furos detectados:", len(furos))
```

Momentos: de onde saem área, centróide e orientação

UNIDADE 2.2

Momentos são somas ponderadas das coordenadas dos pixels de uma região.
Apesar do nome intimidador, eles são a origem das medidas mais úteis do pipeline.

Definição geral

$$m(p,q) = \sum \sum x^p \cdot y^q \cdot I(x,y)$$

$$\begin{aligned} \text{Área} &= m_{00} \\ \text{Centróide} &= (m_{10} / m_{00}, m_{01} / m_{00}) \end{aligned}$$

Momentos centrais (μ) subtraem o centróide e ficam invariantes à translação.

MOMENTOS E MEDIDAS BÁSICAS

```
M = cv2.moments(c)
area = M["m00"]
cx = int(M["m10"] / M["m00"])
cy = int(M["m01"] / M["m00"])

# area tambem pode vir direto:
area_alt = cv2.contourArea(c)
perimetro = cv2.arcLength(c, True) # True = fechada
```

Descritor	Função OpenCV	O que informa
Área	cv2.contourArea(c)	Tamanho em pixels — base para filtrar ruído por área mínima
Perímetro	cv2.arcLength(c, True)	Comprimento da fronteira — entra no cálculo de circularidade
Caixa alinhada	cv2.boundingRect(c)	x, y, largura e altura — recorte rápido da ROI
Caixa rotacionada	cv2.minAreaRect(c)	Centro, dimensões e ângulo — mede a orientação real da peça
Círculo mínimo	cv2.minEnclosingCircle(c)	Centro e raio — útil para objetos aproximadamente redondos
Envoltória convexa	cv2.convexHull(c)	Menor polígono convexo que contém o objeto — base da solidez

Classificar formas sem inteligência artificial

Combinando os descritores anteriores em razões adimensionais, você obtém números que não dependem do tamanho do objeto na imagem. É assim que sistemas industriais separam categorias simples com poucas linhas de código e desempenho determinístico.

Métrica	Cálculo	Leitura do valor
Circularidade	$4\pi \cdot \text{área} / \text{perímetro}^2$	Vale 1 para o círculo perfeito. Abaixo de 0,80 sugere forma alongada ou borda irregular
Solidez	$\text{área} / \text{área da envoltória convexa}$	Próximo de 1 indica objeto compacto. Valores baixos denunciam reentrâncias — trincas, mordidas, quebras
Extensão (extent)	$\text{área} / \text{área da caixa delimitadora}$	Alta para retângulos, baixa para formas em L, X ou estrela
Razão de aspecto	$\text{largura} / \text{altura da caixa}$	Distingue peças alongadas de peças quadradas independentemente da escala
Diâmetro equivalente	$\sqrt{4 \cdot \text{área} / \pi}$	Converte área em uma dimensão linear comparável entre objetos

CLASSIFICAÇÃO POR REGRAS GEOMÉTRICAS

```
area = cv2.contourArea(c); perim = cv2.arcLength(c, True)
circularidade = 4 * np.pi * area / (perim ** 2 + 1e-6)    # 1e-6 evita divisao por zero
solidez       = area / (cv2.contourArea(cv2.convexHull(c)) + 1e-6)
classe = "moeda" if circularidade > 0.85 else "porca" if solidez < 0.92 else "outro"
```

Antes de treinar uma CNN nas Trilhas 4 e 5, verifique se três razões geométricas já resolvem seu problema. Muitas vezes resolvem — com 100% de explicabilidade.

Momentos de Hu: reconhecer forma sob rotação e escala

UNIDADE 2.2

Propostos por Ming-Kuei Hu em 1962, são sete combinações dos momentos centrais normalizados que permanecem praticamente constantes quando o objeto é transladado, redimensionado ou rotacionado. É um descritor de forma clássico e ainda muito usado em inspeção e OCR de símbolos.

Invariante a translação

Vêm dos momentos centrais, que já descontam o centróide.

Invariante a escala

A normalização pelo momento de ordem zero remove o efeito do tamanho.

Invariante a rotação

As sete combinações foram construídas justamente para cancelar o ângulo.

O sétimo é especial

Muda de sinal sob reflexão — serve para distinguir uma forma de sua imagem espelhada.

MOMENTOS DE HU E COMPARAÇÃO DE FORMAS

```
hu = cv2.HuMoments(cv2.moments(c)).flatten()
# valores variam por varias ordens de grandeza: aplique log para comparar
hu_log = -np.sign(hu) * np.log10(np.abs(hu) + 1e-30)

# comparacao direta entre duas formas:
d = cv2.matchShapes(c1, c2, cv2.CONTOURS_MATCH_I1, 0.0) # 0 = formas
identicas
```

Limites honestos do método

Momentos de Hu descrevem a forma global. Eles não distinguem objetos que diferem por textura ou por detalhes finos, e são sensíveis a ruído no contorno e a oclusão parcial. Para esses casos, os descritores aprendidos das Trilhas 4 e 5 são superiores.

Referência: HU, M.-K. Visual pattern recognition by moment invariants. IRE Transactions on Information Theory, v. 8, n. 2, p. 179-187, 1962.

Aplicação integrada: contar e medir objetos

CONTAGEM, MEDIÇÃO E ANOTAÇÃO VISUAL

```
contornos, _ = cv2.findContours(limpa, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

AREA_MINIMA = 300          # descarta ruído residual — calibre pelo seu dataset
resultado    = img.copy()
relatorio    = []

for i, c in enumerate(contornos):
    area = cv2.contourArea(c)
    if area < AREA_MINIMA:
        continue

    perim = cv2.arcLength(c, True)
    circ  = 4 * np.pi * area / (perim ** 2 + 1e-6)
    M      = cv2.moments(c)
    cx, cy = int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"])
    x, y, w, h = cv2.boundingRect(c)

    cv2.drawContours(resultado, [c], -1, (0, 255, 0), 2)
    cv2.circle(resultado, (cx, cy), 4, (0, 0, 255), -1)
    cv2.putText(resultado, str(len(relatorio) + 1), (x, y - 8),
                  cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)

    relatorio.append({"id": len(relatorio) + 1, "area": area,
                     "circularidade": round(circ, 3), "centro": (cx, cy)})

print(f"Total de objetos validos: {len(relatorio)}")
```

O filtro por área é obrigatório

Sem ele, qualquer resíduo da binarização entra na contagem. O valor deve ser justificado pela menor peça real do dataset.

Guarde os resultados em lista

Uma lista de dicionários vira DataFrame com uma linha de código e produz as tabelas do relatório técnico.

Desenhe sempre a saída

A imagem anotada é a evidência visual mais forte na defesa do PP1 e revela erros que o número sozinho esconde.

Converta pixels em milímetros

Com um objeto de referência de tamanho conhecido na cena, área em pixels vira área real — e o projeto ganha valor de engenharia.

Transformada de Hough: procurar a forma, não o pixel

UNIDADE 2.2

Contornos funcionam quando o objeto é uma mancha fechada. Mas e quando você procura uma reta — a borda de um documento, a linha de uma via, o eixo de uma peça — e ela aparece quebrada em vários pedaços? A Transformada de Hough resolve isso mudando o espaço em que a busca acontece.

1 · O problema

No espaço da imagem, uma reta interrompida é vários segmentos desconexos. Nenhum algoritmo local os une.

2 · A troca de espaço

Cada pixel de borda "vota" em todas as retas que poderiam passar por ele. Os votos vão para um acumulador indexado por parâmetros da reta.

3 · O pico

Pixels que pertencem à mesma reta votam todos na mesma célula do acumulador. Essa célula acumula um pico — a reta detectada.

4 · A vantagem

Como cada pixel vota independentemente, falhas e oclusões parciais não impedem a detecção. É um método robusto por construção.

A parametrização polar (e por que não usar $y = ax + b$)

$$\rho = x \cdot \cos \theta + y \cdot \sin \theta$$

A forma $y = ax + b$ tem coeficiente angular infinito em retas verticais, o que quebra o acumulador. Na forma polar, ρ é a distância da reta à origem e θ o seu ângulo — ambos limitados e discretizáveis.

Acumulador (ρ, θ) : o pico é a reta



célula em destaque = mais votos = reta mais provável

HoughLinesP na prática: os parâmetros que importam

Parâmetro	Significado	Sintoma quando está errado
rho	Resolução da distância no acumulador, em pixels (use 1)	Valores altos fundem retas próximas em uma só
theta	Resolução angular (use np.pi/180, ou seja, 1 grau)	Valores grosseiros perdem retas levemente inclinadas
threshold	Número mínimo de votos para aceitar uma reta	Baixo demais: dezenas de retas falsas. Alto demais: nenhuma reta
minLineLength	Comprimento mínimo do segmento aceito	Baixo demais: fragmentos de textura viram retas
maxLineGap	Falha máxima tolerada dentro de uma mesma reta	Baixo demais: uma reta tracejada vira vários segmentos

DETECÇÃO DE SEGMENTOS DE RETA

```
bordas = cv2.Canny(cinza, 50, 150, apertureSize=3)
# Hough opera sobre bordas
retas = cv2.HoughLinesP(bordas, 1, np.pi / 180, threshold=80,
                        minLineLength=60, maxLineGap=10)
for x1, y1, x2, y2 in retas[:, 0]:
    cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

HoughLines × HoughLinesP

A versão clássica (HoughLines) devolve retas infinitas em (ρ, θ) — você precisa calcular os pontos para desenhar. A versão probabilística (HoughLinesP) devolve diretamente os extremos dos segmentos e é mais rápida. Para aplicações práticas, use a probabilística.

HoughCircles: detectar círculos sem segmentar/

UNIDADE 2.2

A versão para círculos usa um acumulador tridimensional (centro x, centro y, raio). Como isso é caro, o OpenCV implementa o gradiente de Hough, que estima primeiro os centros e só depois os raios.

Parâmetro	O que controla	Orientação de ajuste
dp	Razão inversa da resolução do acumulador	Comece com 1.2; aumente se nenhum círculo for detectado
minDist	Distância mínima entre centros detectados	Use algo próximo ao diâmetro do menor objeto, para evitar detecções duplicadas
param1	Limiar superior do Canny interno	Mesma lógica de ajuste do Canny da Trilha 1
param2	Limiar do acumulador para aceitar um círculo	O parâmetro mais sensível: baixo gera círculos fantasma, alto não detecta nada
minRadius / maxRadius	Faixa de raios pesquisada	Restringir a faixa acelera muito e reduz falsos positivos

DETECÇÃO DE CÍRCULOS

```
suave = cv2.medianBlur(cinza, 5)          # mediana funciona melhor que gaussiano aqui
circ  = cv2.HoughCircles(suave, cv2.HOUGH_GRADIENT, dp=1.2, minDist=40,
                        param1=120, param2=45, minRadius=15, maxRadius=90)

if circ is not None:
    for x, y, r in np.uint16(np.around(circ))[0]:
        cv2.circle(img, (x, y), r, (0, 255, 0), 2)
```

Sempre teste "if circ is not None": quando nada é detectado, a função devolve None e o laço quebra com TypeError.

Retificação de perspectiva: a foto vira scanner

UNIDADE 2.2

Uma foto de documento tirada de lado tem seus cantos deformados por projeção perspectiva.

A homografia é a matriz 3×3 que mapeia quatro pontos da imagem original para quatro pontos de destino, desfazendo essa deformação.

1 Segmentar o documento

Cinza → blur → Canny (ou Otsu) → fechamento morfológico para unir a borda

2 Achar o maior contorno

findContours + max por área: o documento costuma ser o maior objeto da cena

3 Aproximar para 4 vértices

cv2.approxPolyDP com epsilon = $0,02 \times$ perímetro reduz o contorno a um quadrilátero

4 Ordenar os cantos

Superior-esquerdo, superior-direito, inferior-direito, inferior-esquerdo — a ordem precisa ser consistente

5 Calcular a homografia

cv2.getPerspectiveTransform(origem, destino) devolve a matriz 3×3

6 Aplicar a transformação

cv2.warpPerspective produz a imagem retificada, pronta para OCR

PIPELINE DE SCANNER INTELIGENTE

```
peri = cv2.arcLength(maior, True)
aprox = cv2.approxPolyDP(maior, 0.02 * peri, True) # espera-se len(aprox) == 4
H = cv2.getPerspectiveTransform(np.float32(cantos_ordenados), destino)
plano = cv2.warpPerspective(img, H, (largura_final, altura_final))
```

Erros que mais aparecem na Unidade 2.2

UNIDADE 2.2

! Contar sem filtrar por área

Ruído residual vira objeto. Sempre defina `AREA_MINIMA` a partir da menor peça real, medida em pixels.

! Passar o Canny para `findContours`

Gera contornos duplicados — um interno e outro externo à linha de borda — e dobra a contagem.

! Esquecer que `HoughCircles` pode devolver `None`

O código quebra com `TypeError` na primeira imagem em que nada é detectado.

! Não ordenar os cantos na homografia

A imagem retificada sai espelhada ou girada 90°, e o OCR falha sem motivo aparente.

! Comparar áreas entre imagens de resoluções diferentes

Área em pixels depende da resolução. Normalize ou converta para unidade física antes de comparar.

! Ajustar parâmetros do Hough no olho

Faça uma varredura sistemática de `threshold` e `minLineLength` e registre no relatório qual faixa funcionou.

2.3

Domínio da Frequência e Contornos Ativos

Duas maneiras diferentes de olhar a mesma imagem: como soma de ondas e como curva que se deforma

Fourier

DFT, espectro e filtragem em frequência

Wavelets

Análise multiescala e localização

Snakes

Contornos ativos e modelos deformáveis

Trocar de lente: o que é o domínio da frequência

UNIDADE 2.3

Até aqui a imagem foi tratada como uma grade de intensidades. A Transformada de Fourier propõe outra descrição da mesma informação: qualquer imagem pode ser escrita como uma soma de padrões ondulatórios de diferentes frequências, orientações e intensidades.

A analogia do equalizador

Um equalizador de áudio não altera a música nota por nota: ele separa graves, médios e agudos e ajusta cada faixa. A DFT faz o mesmo com uma imagem — separa "graves" (variações lentas: fundos, sombras, iluminação geral) de "agudos" (variações rápidas: bordas, texturas, ruído). Filtrar em frequência é mexer nesses controles.

Baixas frequências

Centro do espectro

Iluminação geral, regiões amplas e uniformes, a "silhueta" da imagem

Altas frequências

Periferia do espectro

Bordas, texturas finas, detalhes — e também o ruído

Picos isolados

Pontos brilhantes fora do centro

Padrões periódicos: listras de digitalização, moiré, interferência de sensor

Por que isso é útil na Visão Computacional

Três motivos concretos. (1) Diagnóstico: o espectro revela de imediato se existe ruído periódico, borramento direcional ou aliasing na aquisição. (2) Filtragem seletiva: é possível remover exatamente uma faixa de frequência sem tocar no resto — algo difícil de fazer com kernels espaciais. (3) Desempenho: pelo teorema da convolução, filtrar com kernels muito grandes sai mais barato multiplicando no domínio da frequência do que convoluindo no espacial.

DFT em imagens: magnitude, fase e o espectro

UNIDADE 2.3

Espectro de magnitude

Diz QUANTO de cada frequência existe na imagem. É o que se costuma visualizar como uma "estrela" clara no centro sobre fundo escuro.

Espectro de fase

Diz ONDE cada componente está posicionada. Menos vistoso, porém é a fase que carrega a estrutura reconhecível da cena.

fftshift

A DFT coloca a frequência zero no canto superior esquerdo. O `fftshift` move essa origem para o centro, tornando o espectro legível.

Escala logarítmica

A componente central é ordens de grandeza maior que as demais. Sem log, você vê apenas um ponto branco e nada mais.

CALCULANDO E VISUALIZANDO O ESPECTRO

```
f = np.fft.fft2(cinza) # transformada 2D
fshift = np.fft.fftshift(f) # frequencia zero para o centro
magnitudo = 20 * np.log(np.abs(fshift) + 1) # escala log para visualizar

plt.subplot(121); plt.imshow(cinza, cmap="gray"); plt.title("Imagem")
plt.subplot(122); plt.imshow(magnitudo, cmap="gray"); plt.title("Espectro")

# alternativa com OpenCV (mais rapida em imagens grandes):
# dft = cv2.dft(np.float32(cinza), flags=cv2.DFT_COMPLEX_OUTPUT)
```

Como ler um espectro

Linhas claras no espectro aparecem perpendiculares às bordas dominantes da imagem: uma cena com muitas linhas verticais produz um espectro com energia espalhada na horizontal. Pontos brilhantes simétricos fora do centro indicam um padrão periódico — quase sempre ruído de aquisição a ser removido.

Filtrar no espectro: o teorema da convolução

PASSA-BAIXA

Zera a periferia, mantém o centro

Efeito: borra a imagem. Equivale a um filtro de suavização espacial, porém com controle exato da frequência de corte.

PASSA-ALTA

Zera o centro, mantém a periferia

Efeito: realça bordas e remove iluminação de fundo. Base conceitual dos realces de contorno.

REJEITA-FAIXA (notch)

Zera pontos específicos do espectro

Efeito: elimina exatamente um padrão periódico — listras, moiré, interferência — sem borrar o resto da imagem.

Teorema da convolução

$$f(x, y) * h(x, y) \leftrightarrow F(u, v) \cdot H(u, v)$$

Convoluir no espaço equivale a multiplicar em frequência. Esse é o resultado que conecta toda a Trilha 1 a esta unidade: os filtros de média, gaussiano e Sobel que você aplicou como kernels são, no domínio da frequência, simples máscaras multiplicativas. Para kernels grandes, a rota pela FFT chega a ser ordens de grandeza mais rápida.

FILTRO PASSA-BAIXA IDEAL

```
linhas, cols = cinza.shape
cr, cc = linhas // 2, cols // 2
R = 30                                     # raio de corte

mascara = np.zeros((linhas, cols), np.uint8)
cv2.circle(mascara, (cc, cr), R, 1, -1)
# 1 no centro = passa-baixa

filtrado = np.fft.ifft2(np.fft.ifftshift(fshift * mascara))
saida = np.abs(filtrado)                   # volta ao dominio
espacial
```

Atenção: máscaras com bordas abruptas (o "filtro ideal") produzem artefatos de ondulação chamados ringing. Na prática usam-se transições suaves, como o filtro de Butterworth ou uma máscara gaussiana.

Ruído periódico: quando a frequência vence

UNIDADE 2.3

Digitalizações de documentos antigos, imagens de sensores com interferência elétrica e fotos de telas apresentam padrões repetitivos de listras. No domínio espacial, suavizar para removê-los destrói também as bordas úteis. No domínio da frequência, esse ruído aparece como poucos pontos isolados — e pode ser eliminado cirurgicamente.

1

Calcular o espectro

FFT + fftshift + escala
logarítmica

2

Localizar os picos

Pontos brilhantes
simétricos fora do centro

3

Zerar com máscara notch

Pequenos discos de valor
zero sobre cada pico

4

Transformada inversa

ifftshift + ifft2 + módulo

5

Validar

Comparar histograma e
bordas antes e depois

Contexto de mercado

Bibliotecas de restauração de acervos digitais e softwares de imagem médica usam exatamente esta rotina. É também um exemplo excelente para o relatório do PP1: mostra domínio técnico e produz um "antes e depois" visualmente convincente.

Limite honesto do método

Só funciona quando o ruído é realmente periódico. Ruído gaussiano e sal-e-pimenta se espalham por todo o espectro e continuam sendo tratados melhor no domínio espacial, com filtro gaussiano e mediana.

```
notch = np.ones(fshift.shape, np.float32)
for (px, py) in picos_detectados:           # coordenadas dos picos no espectro
    cv2.circle(notch, (px, py), 6, 0, -1)   # anula uma pequena vizinhança
```

Wavelets: onde a Transformada de Fourier não alcança

UNIDADE 2.3

A DFT informa quais frequências existem na imagem, mas não onde elas ocorrem: a informação espacial é diluída em todo o espectro. Wavelets resolvem esse problema analisando a imagem em várias escalas simultaneamente, preservando a localização de cada componente.

Aspecto	Fourier (DFT)	Wavelets
Base de decomposição	Senos e cossenos infinitos	Ondaletas curtas e localizadas (Haar, Daubechies, Coiflet)
Localização espacial	Perdida — vale para a imagem inteira	Preservada — sabe-se onde cada detalhe está
Análise de escala	Uma única resolução de frequência	Multiescala por construção: aproximação + detalhes em vários níveis
Uso típico	Diagnóstico espectral e filtragem periódica	Compressão (JPEG 2000), remoção de ruído, detecção de defeitos locais

A decomposição em quatro sub-bandas

Um nível de decomposição 2D divide a imagem em: LL (aproximação, versão reduzida da imagem), LH (detalhes horizontais), HL (detalhes verticais) e HH (detalhes diagonais). Cada novo nível repete o processo sobre a sub-banda LL.

DECOMPOSIÇÃO WAVELET EM UM NÍVEL

```
import pywt # pip install PyWavelet
coef = pywt.dwt2(cinza, "haar")
LL, (LH, HL, HH) = coef

# reconstrucao (por exemplo, apos zerar detalhes ruidosos):
reconstruida = pywt.idwt2((LL, (LH, HL, HH)), "haar")
```

Contornos Ativos (Snakes): a curva que se molda

UNIDADE 2.3

Propostos por Kass, Witkin e Terzopoulos em 1988, os contornos ativos partem de uma curva inicial desenhada perto do objeto e a deformam iterativamente até que ela se acomode sobre a fronteira. Em vez de decidir pixel a pixel, o método otimiza a curva inteira.

ENERGIA INTERNA

Depende só da forma da curva

- Controlada por α — elasticidade: penaliza curvas longas, faz a snake contrair.
- Controlada por β — rigidez: penaliza dobras bruscas, mantém a curva suave.
- Sozinha, faria a curva encolher até virar um ponto.

ENERGIA EXTERNA

Depende da imagem

- Normalmente derivada do gradiente: a curva é atraída por regiões de borda forte.
- Pode ser ajustada para atrair a curva a regiões claras ou escuras.
- Sozinha, faria a curva se prender a qualquer ruído com gradiente alto.

O equilíbrio entre as duas energias é o que define o resultado: a snake para onde a atração da imagem se iguala à resistência da própria curva.

CONTORNO ATIVO COM SCIKIT-IMAGE

```
from skimage.segmentation import active_contour
snake = active_contour(gaussian(cinza, 3), curva_inicial,
                      alpha=0.015, beta=10, gamma=0.001)
```

A limitação decisiva

A snake depende fortemente da curva inicial. Se ela começar longe do objeto ou dentro de outra estrutura, converge para o lugar errado. Por isso costuma ser usada em fluxos semiautomáticos, com o usuário indicando a posição inicial.

Representação interna: a ponte para a Trilha 3

UNIDADE 2.3

Todas as técnicas desta trilha produzem uma descrição interna do objeto — uma forma compacta de guardar "o que ele é" para comparação posterior. Entender essa ideia prepara o próximo tema da disciplina: o problema da rotulagem consistente.

Fronteira

Lista de pontos do contorno, cadeia de códigos, descritores de Fourier da curva

Compacta e boa para forma; sensível a oclusão

Região

Máscara, área, momentos, momentos de Hu, esqueleto morfológico

Robusta a pequenas falhas de borda; ignora detalhe interno

Relacional

Grafo de partes: quais componentes existem e como se conectam

Base do Consistent Labelling — próximo tema da disciplina

Aparência

Padrões de intensidade aprendidos a partir de exemplos

É o caminho das CNNs, nas Trilhas 4 e 5

O que vem na Trilha 3

A rotulagem consistente (Consistent Labelling) pergunta: dado um conjunto de partes detectadas e um conjunto de restrições entre elas, qual atribuição de rótulos é simultaneamente compatível com todas as restrições? É a formalização de "esta região é a cabeça do parafuso porque está ligada a esta outra região que é a rosca". Os descritores que você aprendeu aqui são exatamente as evidências que alimentam esse raciocínio.

Erros que mais aparecem na Unidade 2.3

UNIDADE 2.3

! Visualizar o espectro sem escala log

Aparece apenas um ponto branco no centro e a conclusão errada de que "a FFT não funcionou".

! Esquecer o fftshift antes de mascarar

A máscara circular acaba centrada no lugar errado e o filtro faz o oposto do pretendido.

! Usar filtro ideal e estranhar as ondulações

Cortes abruptos no espectro geram ringing. Use transições suaves (gaussiana ou Butterworth).

! Tentar remover ruído gaussiano com notch

Ruído não periódico se espalha por todo o espectro; notch só resolve padrões repetitivos.

! Inicializar a snake longe do objeto

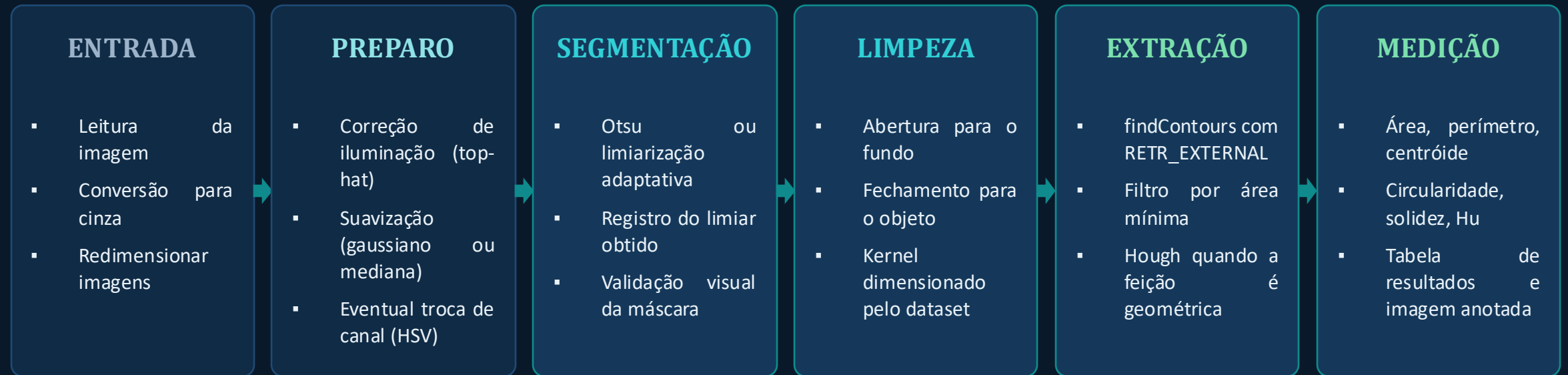
O contorno ativo converge para o mínimo de energia mais próximo, que pode não ser a fronteira desejada.

! Tratar Wavelets como se fossem obrigatórias

Nesta disciplina são introdutórias. Use-as no projeto apenas se houver justificativa técnica clara.

O pipeline completo da Trilha 2 em uma página

SÍNTESE



Documente cada decisão

Para cada etapa, o relatório técnico deve responder: qual parâmetro foi usado, por que esse valor e o que muda se ele for alterado. É essa justificativa que separa um projeto de nota máxima de um script que "funcionou".

Modularize desde já

Transforme cada fase em uma função com entrada e saída explícitas. Nas Trilhas 4 e 5 esse mesmo pipeline vira o pré-processamento do modelo de Deep Learning — e código organizado agora poupa retrabalho depois.

Qual técnica usar para qual problema

Problema real	Caminho recomendado	Armadilha a evitar
Contar peças em fundo uniforme	Otsu → abertura → findContours + filtro de área	Esquecer o filtro de área e contar ruído
Fotografia de documento com sombra	Adaptativa gaussiana → fechamento → contorno → homografia	Insistir em limiar global e culpar a câmera
Detectar borda de estrada ou trilho	Canny → HoughLinesP com faixa angular restrita	Aceitar todas as retas, inclusive as do horizonte
Contar objetos circulares sobrepostos	HoughCircles ou watershed sobre a transformada de distância	Usar contornos, que fundem objetos encostados
Inspecionar trinca em superfície metálica	Top-hat → realce de contraste → morfologia direcional → solidez	Suavizar demais e apagar exatamente a trinca
Remover listras de digitalização	FFT → identificar picos → máscara notch → inversa	Tentar resolver com filtro de mediana
Medir orientação de uma peça	Contorno → cv2.minAreaRect → ângulo retornado	Usar boundingRect, que ignora rotação

Nenhuma dessas soluções exige treinamento de modelo.

Todas produzem resultado determinístico, auditável e executável em tempo real — inclusive em hardware modesto.

Onde isso aparece fora da sala de aula

APLICAÇÕES

Manufatura e controle de qualidade

Sistemas de inspeção em linha usam limiarização, morfologia e descritores de forma para verificar presença de furos, integridade de solda e conformidade dimensional em milissegundos por peça.

Documentos e OCR

Todo aplicativo de digitalização por celular executa a mesma sequência: detectar bordas, achar o quadrilátero, aplicar homografia, binarizar de forma adaptativa e só então rodar o reconhecimento de caracteres.

Imagem médica e laboratorial

Contagem de células, medição de áreas de lesão e segmentação de estruturas em lâminas continuam apoiadas em Otsu, morfologia, watershed e contornos ativos — muitas vezes combinados com redes neurais.

Mobilidade e monitoramento

Deteção de faixas de rolamento, leitura de placas e contagem de veículos partem de Hough e retificação de perspectiva antes de qualquer classificador entrar em cena.

Em todos esses domínios, o processamento clássico não foi substituído pelo Deep Learning: ele virou a camada de preparo que torna o modelo viável, mais rápido e mais barato de operar.

Resumo: dez ideias para levar da Trilha 2

FECHAMENTO

1 Binarizar é tomar uma decisão de engenharia; o histograma é o instrumento que fundamenta essa decisão.

2 Otsu automatiza a escolha do limiar maximizando a separação entre as duas populações de pixels — e exige histograma bimodal.

3 Iluminação irregular não se resolve com limiar global: use limiarização adaptativa ou corrija o fundo com top-hat.

4 Morfologia trata a máscara como conjunto: abertura limpa o fundo, fechamento tapa buracos no objeto.

5 O tamanho do elemento estruturante deve ser maior que o ruído e menor que a menor feição relevante.

6 Borda e contorno são coisas distintas: findContours precisa de máscara binária, não do resultado do Canny.

7 A hierarquia de contornos revela furos e estruturas aninhadas — informação de inspeção que sai de graça.

8 Circularidade, solidez e momentos de Hu classificam formas simples sem nenhum treinamento de modelo.

9 Hough encontra formas geométricas mesmo interrompidas, porque cada pixel vota de forma independente.

10 O domínio da frequência diagnostica a aquisição e remove ruído periódico; Wavelets acrescentam localização espacial.

Conexão com a AED — Checkpoint 1 em 11/09/2026

PROJETO INTEGRADOR

A Trilha 2 coincide exatamente com o Checkpoint 1 do Projeto Integrador.

O que você estudou aqui é, literalmente, o que deve estar entregue no repositório da equipe.

Dataset estruturado

Imagens organizadas em pastas, quantidade e dimensões documentadas, fontes citadas (Roboflow, Kaggle, Open Images ou coleta própria).

Pipeline de pré-processamento

Código que lê, redimensiona, converte espaço de cor, filtra ruído, segmenta e limpa a máscara — com parâmetros justificados.

Arquitetura da solução

Diagrama do fluxo de dados: da entrada até a saída pretendida, indicando onde o modelo de IA entrará nas próximas trilhas.

Evidências visuais

Comparações "antes e depois" de cada etapa. São elas que sustentam a defesa técnica do PP1 em 29/09 e 02/10.

Calendário próximo da AED

Checkpoint 1 em 11/09/2026: repositório com dataset estruturado e pipeline de pré-processamento funcionando. · Apresentação técnica do PP1 nos dias 29/09 (Grupo A) e 02/10 (Grupo B), valendo até 4,0 pontos na N1. · Atividade Estruturada 1 em 25/09/2026, valendo até 2,0 pontos, cobrando filtragem, segmentação por Otsu, morfologia e contornos — exatamente o conteúdo desta trilha.

Autoavaliação: você consegue responder sem consultar?

ESTUDO

01 Por que o método de Otsu exige um histograma bimodal, e o que acontece quando essa condição não é satisfeita?

02 Qual a diferença prática entre aplicar abertura e aplicar fechamento em uma máscara com ruído no fundo?

03 Por que passar o resultado do Canny para findContours costuma duplicar a contagem de objetos?

04 O que os quatro números do vetor de hierarquia de um contorno representam?

05 Como circularidade e solidez podem separar uma moeda de uma porca sextavada sem nenhum modelo treinado?

06 Por que a Transformada de Hough usa a parametrização polar (ρ, θ) em vez de $y = ax + b$?

07 O que significa um par de pontos brilhantes simétricos fora do centro no espectro de Fourier?

08 Qual limitação das Wavelets resolve em relação à Transformada de Fourier, e a que custo?

Se travar em alguma delas, volte à unidade correspondente antes de seguir para a Trilha 3 — e traga a dúvida para o Fórum da disciplina.

Referências e material de aprofundamento

Bibliografia

- FORSYTH, D.; PONCE, J. Computer vision: a modern approach. 2. ed. Boston: Pearson, 2012.
- SHAPIRO, L. G.; STOCKMAN, G. C. Computer vision. Upper Saddle River: Prentice Hall, 2001.
- UMBAUGH, S. E. Digital image processing and analysis. 2. ed. Boca Raton: CRC Press, 2011.
- GONZALEZ, R. C.; WOODS, R. E. Digital image processing. 2. ed. Upper Saddle River: Prentice-Hall, 2002. (capítulos 9 e 10)
- PEDRINI, H.; SCHWARTZ, W. R. Análise de imagens digitais. São Paulo: Thomson Learning, 2008.

Artigos originais citados

- OTSU, N. A threshold selection method from gray-level histograms. IEEE Trans. Systems, Man and Cybernetics, v. 9, n. 1, p. 62-66, 1979.
- HU, M.-K. Visual pattern recognition by moment invariants. IRE Trans. Information Theory, v. 8, n. 2, p. 179-187, 1962.
- KASS, M.; WITKIN, A.; TERZOPOULOS, D. Snakes: active contour models. Int. Journal of Computer Vision, v. 1, n. 4, p. 321-331, 1988.
- DUDA, R. O.; HART, P. E. Use of the Hough transformation to detect lines and curves in pictures. Communications of the ACM, v. 15, n. 1, p. 11-15, 1972.

Material aberto recomendado (verificado e gratuito)

- ▶ **First Principles of Computer Vision — Columbia University (Prof. Shree Nayar)**
- ▶ **OpenCV-Python Tutorials — documentação oficial**
- ▶ **3Blue1Brown — But what is the Fourier Transform? A visual introduction**
- ▶ **MARENGONI, M.; STRINGHINI, D. Tutorial: Introdução à Visão Computacional usando OpenCV**

Videoaulas e monografias em PDF, gratuitas. Módulos diretamente relacionados: Binary Images, Boundary Detection e Image Processing II.

fpcv.cs.columbia.edu

Páginas de Image Thresholding, Morphological Transformations, Contours, Hough Line/Circle Transform e Image Transforms.

docs.opencv.org

Introdução visual à Transformada de Fourier, com legendas disponíveis. Excelente antes de estudar a DFT em imagens.

youtube.com/watch?v=sPUNpyF58BY

Tutorial em português publicado na Revista de Informática Teórica e Aplicada (RITA/UFRGS).

seer.ufrgs.br

FIM DA TRILHA 2

Você já sabe transformar uma imagem em números

Segmentar, limpar, extrair e medir. Da próxima vez que abrir uma imagem, você não verá mais uma foto — verá uma matriz com objetos identificáveis e mensuráveis.

A SEGUIR — TRILHA 3

Rotulagem consistente, casamento estrutural e fechamento da 1ª etapa avaliativa

ENTREGAS PRÓXIMAS

Checkpoint 1 da AED (11/09) · Ex1 (25/09) · Defesa do PP1 (29/09 e 02/10) · P1 (06/10)

CANAL DE DÚVIDAS

Fórum de Dúvidas da disciplina no AVA — descreva o erro, cole o código e anexe a imagem de saída